

Exercício de Fixação — Subprogramas

Página / Slide 22

Aluno: Rodrigo Del Padre**RA:** 24022092-2**Curso:** Engenharia de Software (6º Semestre)**Turma:** B**Disciplina:** Paradigmas de Programação**Referência:** R. Sebesta, 11ª ed., Caps. 9 e 10

Trecho 3 — Python

Código analisado:

```
fs = [lambda: i for i in range(3)]  
print([f() for f in fs])
```

(a) Qual é a saída?

```
[2, 2, 2]
```

(b) Qual conceito da aula explica?

Conceito Central: Fechamentos (*Closures*), Escopo Léxico e Captura Tardia (*Late Binding*) (Sebesta, 11ª ed., §9.12, p. 404–406; Slides 16 e 17).

- **Captura por Referência de Variável:** O fechamento retém a célula de memória da variável *i* do ambiente estático, e não o valor escalar pontual que ela possuía no momento da definição da *lambda*.
- **Avaliação Tardia:** As três *lambdas* são instanciadas durante o laço, mas nenhuma é avaliada naquele instante. Todas compartilham a referência léxica da mesma variável *i*.
- **Valor no Término do Laço:** Ao término do laço `for i in range(3)`, a variável *i* permanece com o valor final 2.
- **Execução:** Ao invocar `[f() for f in fs]`, todas as três *lambdas* leem a variável *i* com valor corrente 2, retornando a lista `[2, 2, 2]`.
- **Correção:** Para capturar o valor imediatamente a cada volta: `fs = [lambda i=i: i for i in range(3)]`.

Trecho 4 — C

Código analisado:

```
int contador(void) {  
    static int n = 0;  
    return ++n;  
}  
  
// em main:  
contador();  
contador();  
printf("%d\n", contador());
```

(a) Qual é a saída?

3

(b) Qual conceito da aula explica?

Conceito Central: Ambientes de Referenciamento Local: Variável Local Estática (*Static Local Variable*) vs. Dinâmica da Pilha (Sebesta, 11ª ed., §9.4, p. 373–375; Slide 7).

- **Segmento de Dados Estáticos:** A palavra-chave `static` faz com que `n` não seja alocada no Registro de Ativação transitório na pilha, mas no segmento de dados com *tempo de vida global*, retendo seu estado entre chamadas sucessivas.
- **Escopo Local:** Embora seu tempo de vida dure toda a execução do programa, seu escopo de visibilidade léxica é estritamente privado ao corpo da função `contador`.
- **Inicialização Única:** A inicialização `n = 0` ocorre uma única vez durante a carga do programa (tempo de inicialização estática).
- **Sequência de Execução:**
 - 1ª chamada: `++n` vai para 1;
 - 2ª chamada: `++n` vai para 2;
 - 3ª chamada (no argumento de `printf`): `++n` vai para 3 e retorna 3, exibido no console.